Executive Summary: Crypto Pulse Dashboard

Project goal

Crypto Pulse is a public web application that tracks a curated basket of major cryptocurrencies and summarizes market conditions in a dashboard that is easy to present in class. The system is designed to answer four questions quickly: Which assets are leading, which are weakening, how concentrated the market is, and whether the basket is moving together or diverging. The project intentionally uses a custom web stack instead of business intelligence tools, so it stays within the assignment rules.

Data pipeline and back-end

The back-end is a Python ETL pipeline connected to the CoinGecko public API. In the **extract** stage, the pipeline downloads the latest market snapshot for twelve dashboard assets and thirty days of daily history for the same basket. In the **transform** stage, it calculates 24-hour and 7-day price changes, 30-day returns, 30-day volatility, average trading volume, breadth signals, concentration metrics, and return series for correlation analysis. In the **load** stage, the transformed results are stored in SQLite tables for pipeline runs, latest snapshots, and daily price history. This structure makes the app more than a one-time visualization because the data is cleaned, persisted, and reusable across refreshes.

Dashboard and communication value

The front-end is built with custom HTML, CSS, JavaScript, and Chart.js, not Tableau or Power BI. The dashboard includes KPI cards, a normalized 30-day performance chart, benchmark comparison against Nasdaq and the Dow, an insight panel for breadth and regime signals, a leaders/laggards section, a crypto correlation heatmap, a live news feed, and a detailed asset table. It also includes a pipeline monitor showing run status, refresh timing, record counts, and recent ETL history, which makes the back-end work visible during the demo instead of hiding it behind charts.

Refresh mechanism and deployment

The application supports three refresh behaviors. First, it loads data automatically on startup. Second, it runs scheduled freshness checks in the background and refreshes stale data. Third, it provides a manual refresh button so the pipeline can be triggered live during a presentation. The project is configured for Render deployment, so the final demonstration can use a public URL rather than localhost, satisfying the assignment requirement for a non-local hosted web app.

Grading alignment

This project is strong across the rubric. For **Data Pipeline / ETL / Data Wrangling**, it integrates live API extraction, derived indicators, persistence, and pipeline run logging. For **Visualization**, it combines several coordinated views instead of a single chart. For **Data Refresh Mechanism**, it offers startup seeding, scheduled freshness checks, and manual refresh. For **Communicate**, it includes a concise executive summary, a clean dashboard narrative, and deployment packaging that supports a clear in-class presentation with a real URL.